

Complete Technical Setup & Deployment Manual

This technical guide provides portable, cross-platform instructions to configure, run, and deploy the **Competitor Shift Radar**. Follow these step-by-step instructions to prepare your environment, install dependencies, compile PDF manuals, and publish to GitHub Pages.

Universal Path Notice: All examples throughout this guide use generic paths like `<YOUR_PROJECT_DIRECTORY>` (for example: `C:\projects\competitor-shift-radar` on Windows or `~/projects/competitor-shift-radar` on macOS/Linux). Replace this placeholder with the actual folder on your local machine.

1. Step-by-Step Software Prerequisites

Install the following free, industry-standard tools before running scripts locally:

1 Install Node.js (LTS Version):

- Go to <https://nodejs.org> and download the recommended **LTS (Long Term Support)** installer.
- Run the installer and accept all default settings. Verify in your terminal/PowerShell:

```
node -v      # Should return v18.0.0 or higher (e.g. v20.x, v22.x, v24.x)
npm -v       # Confirms the Node Package Manager is active
```

2 Install Git:

- Download from <https://git-scm.com>. Run the installer with default options.
- Open your terminal and configure your identity (required before your first commit):

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

2. Directory Initialization & Dependency Installation

Create a dedicated folder for your project and install Puppeteer (used to compile PDF manuals and run headless crawlers):

```
# 1. Create and navigate into your project folder
mkdir competitor-shift-radar
cd competitor-shift-radar

# 2. Initialize a standard package.json file
npm init -y

# 3. Install Puppeteer (headless Chrome browser runtime)
npm install puppeteer
```

CommonJS (.cjs) Extension Rule: If your package.json contains "type": "module", scripts using `require()` must end in `.cjs` (e.g., `build_user_manual_pdf.cjs`, `build_technical_manual_pdf.cjs`). This prevents `ReferenceError: require is not defined`.

3. Portable Repository File Tree

<code><YOUR_PROJECT_DIRECTORY>/</code>	
<code> — index.html</code>	<code><- Standalone Interactive Dashboard (0 dependencies)</code>
<code> — build_dossier.cjs</code>	<code><- Optional Headless Web Scraper (Node.js + Puppeteer)</code>
<code> — build_user_manual_pdf.cjs</code>	<code><- Compiles 3-Page User Manual PDF</code>
<code> — build_technical_manual_pdf.cjs</code>	<code><- Compiles 4-Page Technical Manual PDF</code>
<code> — package.json</code>	<code><- Project metadata and npm dependency manifest</code>
<code> — .gitignore</code>	<code><- Excludes node_modules/ from version control</code>
<code> — templates/</code>	
<code> — logo_white.png</code>	<code><- High-resolution brand asset for headers</code>

4. Target Roster In-Memory Schema

The system is streamlined around entity names. The `is_benchmark` flag identifies the primary baseline that all contenders are systematically evaluated against:

```
[
  { "name": "Primary Baseline Subject", "is_benchmark": true },
  { "name": "Contender Alpha", "is_benchmark": false },
  { "name": "Contender Beta", "is_benchmark": false },
  { "name": "Contender Gamma", "is_benchmark": false }
]
```

5. Headless Web Scraper Engine (build_dossier.cjs)

For automated reconnaissance pipelines, this Node.js script reads your roster and uses Puppeteer to simulate browser sessions:

```
const puppeteer = require('puppeteer');
const fs = require('fs');
const path = require('path');

(async () => {
  console.log("Initializing headless competitive crawler...");
  const browser = await puppeteer.launch({ headless: 'new' });
  const page = await browser.newPage();
  await page.setUserAgent('Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36');

  // Roster array can be read from memory or external JSON
  const companies = [
    { name: "Benchmark Baseline", is_benchmark: true },
    { name: "Contender Alpha", is_benchmark: false }
  ];

  let outputSummary = "# Competitive Matrix Dossier\n\n";
  outputSummary += "| Subject | Evaluation Role | Ingestion Status |\n| :--- | :--- | :--- |\n";

  for (const item of companies) {
    const role = item.is_benchmark ? "***Benchmark Baseline**" : "Contender / Target";
    console.log(`Processing: ${item.name}...`);
    outputSummary += `| **${item.name}** | ${role} | Indexed ✓ |\n`;
  }

  await browser.close();
  fs.writeFileSync(path.join(__dirname, 'competitive_matrix.md'), outputSummary);
  console.log("✓ competitive_matrix.md generated successfully.");
})();
```

6. Crucial Git Hygiene: .gitignore

Create a file named `.gitignore` in your root folder. This prevents committing heavy Puppeteer browser binaries:

```
node_modules/
package-lock.json
.env
*.tmp
```

Important: If you omit `node_modules/` from `.gitignore`, Git will attempt to upload tens of thousands of Chromium binary files, resulting in rejected or frozen GitHub pushes.

7. Parameter Ribbon & State Engine (index.html)

The dashboard dynamically manages active comparison dimensions. Selecting or deselecting a parameter pill modifies table rendering and updates the underlying AI prompt:

```
const parameterDefs = {
  general: { id: "general", title: "General Landscape Overview" },
  product: { id: "product", title: "Core Product / Value Prop" },
  target: { id: "target", title: "Target Customer & Positioning" },
  moat: { id: "moat", title: "Observed Moat / Vulnerability" },
  pricing: { id: "pricing", title: "Pricing Model & Tiers" },
  tech: { id: "tech", title: "Tech Stack / Velocity" },
  culture: { id: "culture", title: "Hiring & Culture Signals" }
};

let activeParams = ["general", "product", "target", "moat"];

function toggleParameter(paramKey) {
  if (activeParams.includes(paramKey)) {
    activeParams = activeParams.filter(k => k !== paramKey);
  } else {
    activeParams.push(paramKey);
  }
  if (activeParams.length === 0) activeParams = ["general"];
  renderMatrix();
  generateCustomPrompt();
}
```

8. Gemini 3.8 Flash Direct Browser API Integration

The dashboard calls the modern gemini-3.8-flash endpoint directly from the browser. API keys are stored solely in the user's local browser storage (localStorage) with zero server telemetry:

```
async function runLiveGeminiReport() {
  const apiKey = getApiKey();
  if (!apiKey) return alert("Please provide a valid Google Gemini API Key in the top header.");

  const promptText = generateCustomPrompt();
  const endpoint = `https://generativelanguage.googleapis.com/v1beta/models/gemini-3.8-
flash:generateContent?key=${apiKey}`;

  try {
    const response = await fetch(endpoint, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ contents: [{ parts: [{ text: promptText }] }] })
    });
    const data = await response.json();
    if (data.candidates && data.candidates[0]?.content?.parts[0]?.text) {
      rawReportMarkdown = data.candidates[0].content.parts[0].text;
      document.getElementById('report-output').innerHTML = formatMarkdownToHtml(rawReportMarkdown);
      document.getElementById('report-wrapper').style.display = 'flex';
    } else {
      document.getElementById('report-output').innerText = `API Error: ${data.error?.message || 'Unknown error'}`;
    }
  } catch (err) {
    document.getElementById('report-output').innerText = `Network Error: ${err.message}`;
  }
}
```

9. Client-Side Word (.DOC) & PDF Dossier Serialization

Word exports are generated in memory using standard UTF-8 Blob serialization, embedding an Executive Overview Table, the active Matrix, and formatted AI briefings without server overhead:

```
function downloadReportDoc() {
  const dossierHtml = generateExportDossierHtml();
  const docTemplate = `<html xmlns:o='urn:schemas-microsoft-com:office:office' xmlns:w='urn:schemas-microsoft-com:office:word'>
    <head><meta charset='utf-8'><style>body{font-family:Calibri,Arial;font-size:10pt;line-height:1.5;margin:20mm;}</style></head>
    <body>${dossierHtml}</body></html>`;

  const blob = new Blob(['\u0000', docTemplate], { type: 'application/msword' });
  const url = URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url;
  a.download = `3Sci_Dossier_${Date.now()}.doc`;
  document.body.appendChild(a); a.click(); document.body.removeChild(a);
  URL.revokeObjectURL(url);
}
```

3Sci Open Source Work Hack #4 • Gemini 3.8 Flash & Dossier Export Architecture • 3sci.com

10. Step-by-Step GitHub Setup & Push Instructions

Follow this exact sequence to upload your project to GitHub and obtain a public, shareable evaluation URL:

1 Create an Empty Repository on GitHub:

- Log into <https://github.com> and click **New repository** (or navigate to <https://github.com/new>).
- Enter a repository name (e.g., competitor-shift-radar). Set visibility to **Public**.
- Leave all checkboxes unchecked** (do NOT check "Add a README", .gitignore, or license). Click **Create repository**.

2 Stage, Commit, and Push via Terminal / PowerShell:

```
# 1. Navigate to your project directory
cd <YOUR_PROJECT_DIRECTORY>

# 2. Initialize the Git repository (if not already initialized)
git init

# 3. Stage all modified and new files (excluding node_modules via .gitignore)
git add .

# 4. Create your commit with a descriptive message
git commit -m "Deploy: 18 analysis angles, executive dossier exports, and AI-differentiated matrix"

# 5. Set the default branch name to 'main'
git branch -M main

# 6. Link your local project to your GitHub repository (replace with your actual GitHub URL)
git remote add origin https://github.com/<YOUR_GITHUB_USERNAME>/<YOUR_REPOSITORY_NAME>.git
# (Note: If origin already exists, use: git remote set-url origin
https://github.com/<YOUR_USERNAME>/<YOUR_REPO>.git)

# 7. Push all code to GitHub
git push -u origin main
```

11. Enabling Free Live Hosting (GitHub Pages)

- Navigate to your repository page on GitHub.
- Click the **Settings** tab along the top navigation bar, then select **Pages** in the left sidebar.
- Under **Build and deployment > Source**, ensure **Deploy from a branch** is selected.
- Under **Branch**, select **main** and keep folder as **/ (root)**, then click **Save**.
- Wait 60–90 seconds and refresh the screen. A banner will confirm: **"Your site is live at https://<YOUR_GITHUB_USERNAME>.github.io/<YOUR_REPOSITORY_NAME>"**

12. Technical Verification Matrix

Feature Component	Test Procedure	Expected Verification Result
Top API Key Input	Enter API key in header; refresh browser.	Key persists in localStorage; auto-authenticates AI calls.
AI Matrix Population	Click <i>Populate Matrix with AI</i> .	All cells populate with company-specific intelligence; cells remain editable.
18 Analytical Angles	Switch to <i>Product & Price Comparisons</i> or <i>SWOT</i> .	Prompt studio recalibrates personas, scoring rubrics, and diagnostic questions.
Executive Dossier Export	Click <i>Download Report (.DOC)</i> or <i>(PDF)</i> .	Exports formatted dossier featuring overview table, matrix grid, and narrative.